

TinyWear

Wearable inteligente para
reconocimiento de actividad humana

Curso-taller • ELECTRO 2026

ESP32-S3 + MPU6050 + Wi-Fi directo + UDP unicast + TinyML

Prof. Ernesto Sifuentes de la Hoya

UACJ – Universidad Autónoma de Ciudad Juárez, Chih., México

IIT – Instituto de Ingeniería y Tecnología

Dpto. Ingeniería Eléctrica y Computación



Ruta del curso-taller

Tres días presenciales, un caso integrador y un prototipo que evoluciona en cada sesión.

P1

Día 1- Instrumentar y adquirir

- ❖ Montaje TinyWear
- ❖ MPU6050 e I²C
- ❖ Muestreo a 50 Hz
- ❖ Wi-Fi directo
- ❖ Captura con Python
- ❖ Capturas de prueba

Día 2 - Construir y entrenar

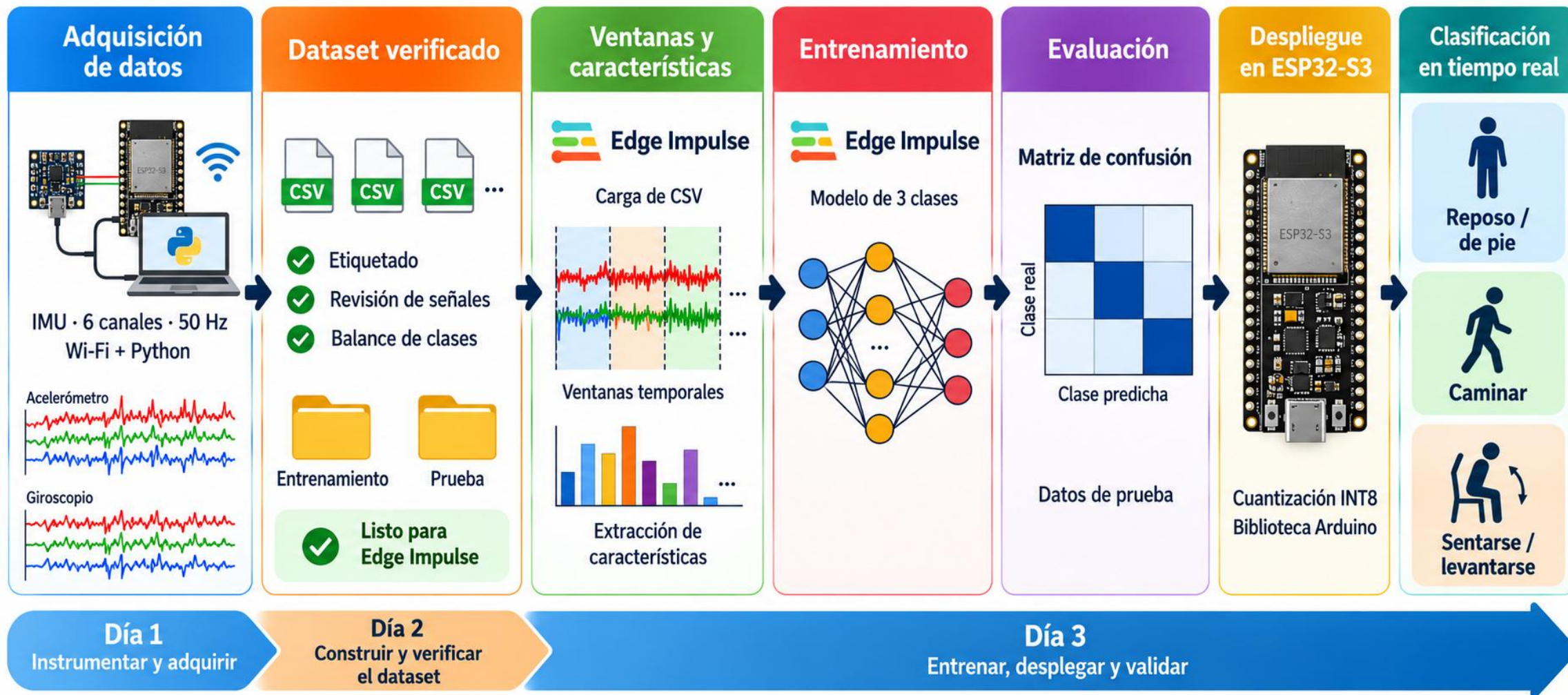
- ❖ Protocolo de las tres actividades
- ❖ Capturas y etiquetado por participante
- ❖ Revisión de señales y Calidad
- ❖ Balance de clases y repeticiones
- ❖ Separación de entrenamiento y prueba
- ❖ Organización y verificación de CSV

Día 3 - Desplegar y validar

- ❖ Carga de CSV en Edge Impulse
- ❖ Ventanas y extracción de características
- ❖ Entrenamiento y matriz de confusión
- ❖ Cuantización y biblioteca Arduino
- ❖ Inferencia en el ESP32-S3
- ❖ Validación y demostración final

Flujo de implementación del TinyWear

Del sensor a la decisión.



Día 1- Instrumentar y adquirir

Meta: Prototipo armado y listo para la captura de datos.

- ✓ ESP32-S3 + MPU6050 + Wi-Fi
- ✓ Montaje en cintura
- ✓ Adquisición a 50 Hz
- ✓ Protocolo de tres actividades
- ✓ Generación de archivos CSV



De medir señales a interpretar estados

La instrumentación inteligente añade una capa de inferencia al proceso de medición.

Instrumentación tradicional

- ❖ Sensor → adquisición → señal → visualización
- ❖ El operador observa y decide
- ❖ Resultado típico: valores, formas de onda y alarmas por umbral

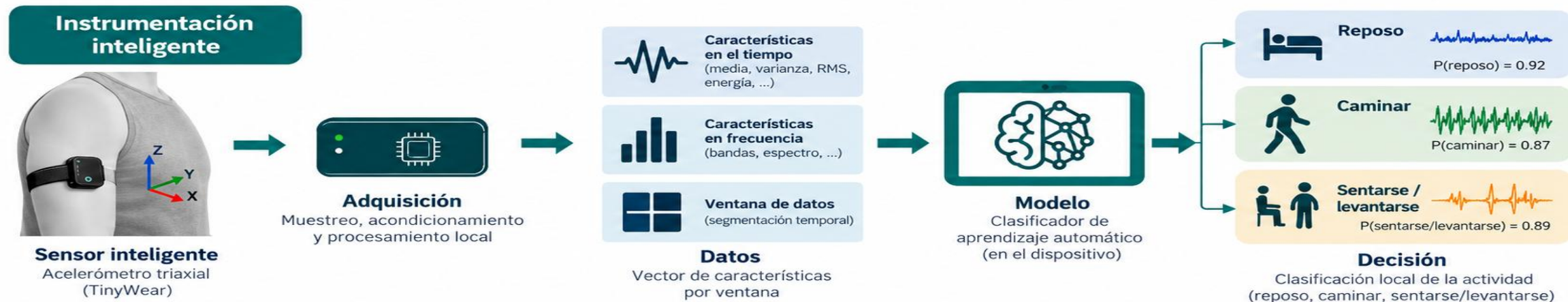
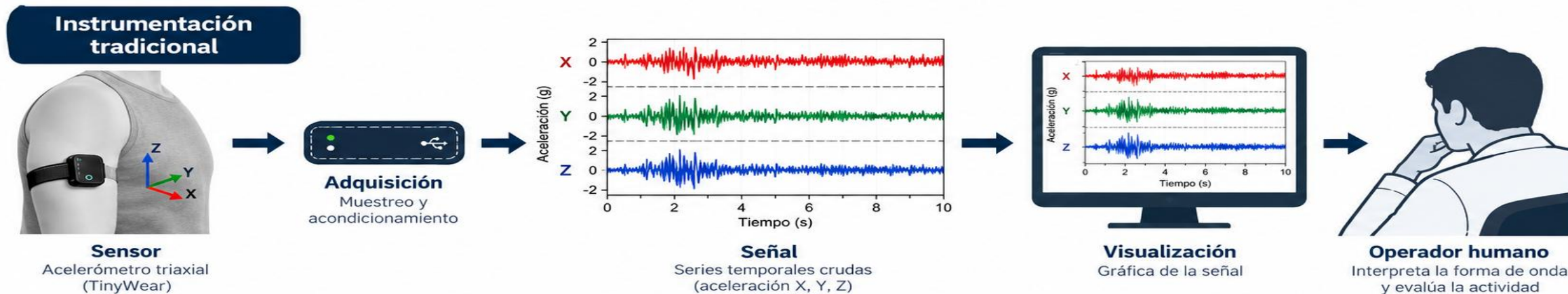


Instrumentación inteligente

- ❖ Sensor → adquisición → datos → modelo → decisión
- ❖ El sistema aprende patrones y clasifica el estado
- ❖ Resultado: información contextual obtenida localmente

De medir señales a interpretar estados

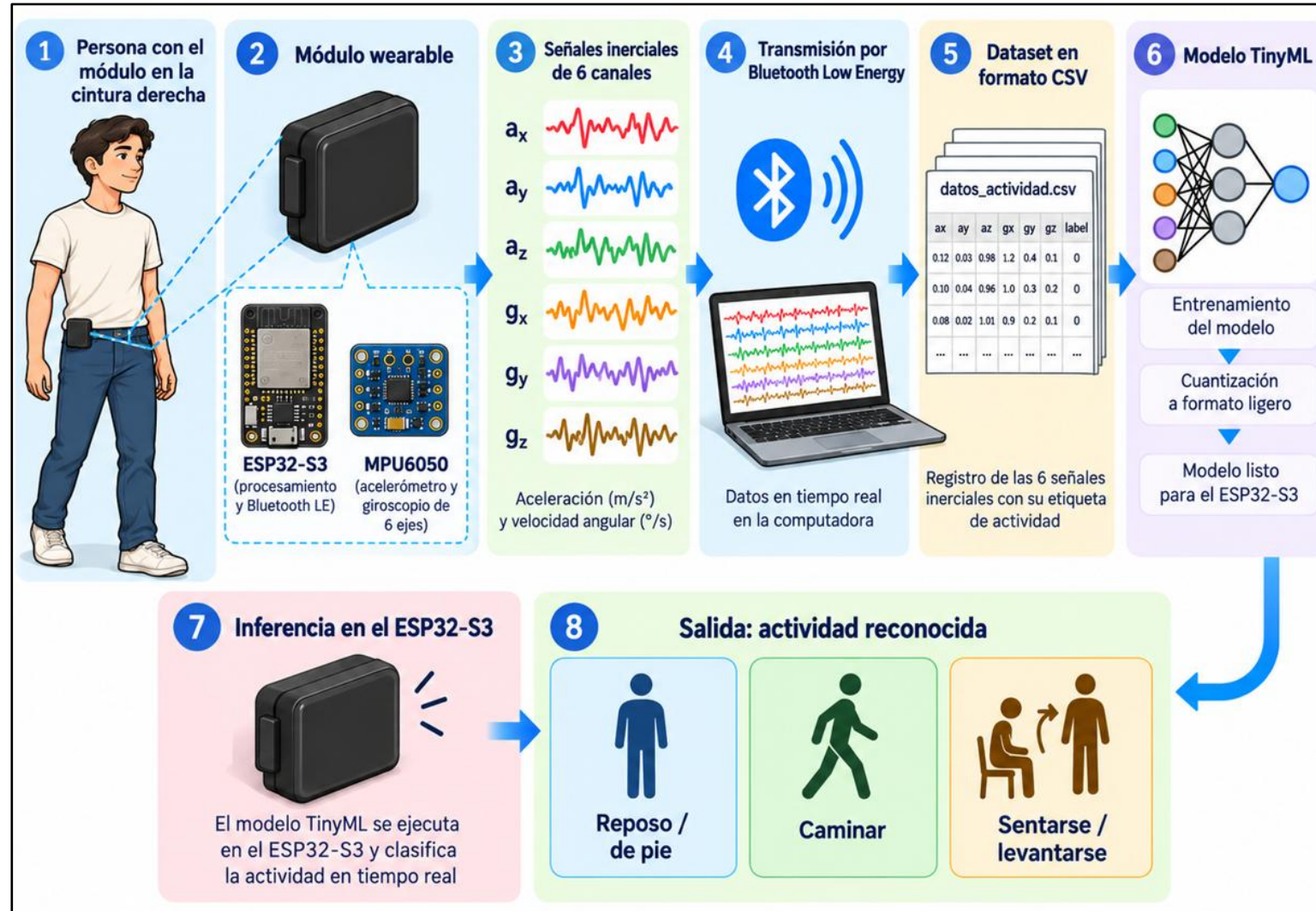
La instrumentación inteligente añade una capa de inferencia al proceso de medición.



TinyWear no solo mide el movimiento: también **interpreta** qué actividad está ocurriendo

Arquitectura de TinyWear

El prototipo separa adquisición, transmisión, entrenamiento y despliegue.



Configuración física del dispositivo

La repetibilidad física también es parte de la calidad del experimento.



Módulo wearable

Clip plástico
Para sujetar en
la cintura

ESP32-S3
(procesamiento
y Bluetooth LE)

MPU6050
(acelerómetro y
giroscopio de 6 ejes)



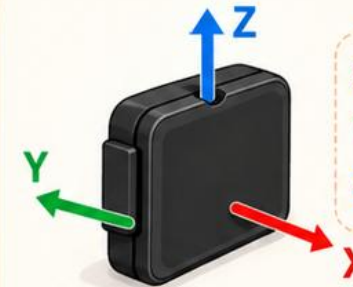
Alimentación portátil
(opcional)

Ubicación del dispositivo



El dispositivo
siempre se coloca
en el **lado derecho
de la cintura.**

Orientación fija



X: hacia el frente
Y: hacia la izquierda
Z: hacia arriba

Buenas prácticas



1. Misma posición

Coloca siempre el dispositivo
en el lado derecho de la cintura.



2. Misma orientación

Mantén la misma orientación
de los ejes (X frente, Y izquierda,
Z arriba).



3. Mismo clip

Usa siempre el mismo
clip plástico para sujetar
el dispositivo.

Hardware y software del sistema

TinyWear integra sensor, microcontrolador, alimentación portátil, Wi-Fi directo y capturador Python.



¿Qué mide el MPU6050?

Seis canales inerciales: tres aceleraciones y tres velocidades angulares.

Acelerómetro

a_x, a_y, a_z

- ❖ Detecta gravedad, inclinación y aceleraciones debidas al movimiento
- ❖ Rango usado: $\pm 4\text{ g}$
- ❖ Escala: 8192 LSB/g

Giroscopio

$\omega_x, \omega_y, \omega_z$

- ❖ Detecta velocidad de rotación y cambios rápidos de orientación
- ❖ Rango usado: $\pm 500\text{ °/s}$
- ❖ Escala: 65.5 LSB/(°/s)

La combinación de ambos sensores ayuda a diferenciar actividades que pueden tener aceleraciones similares

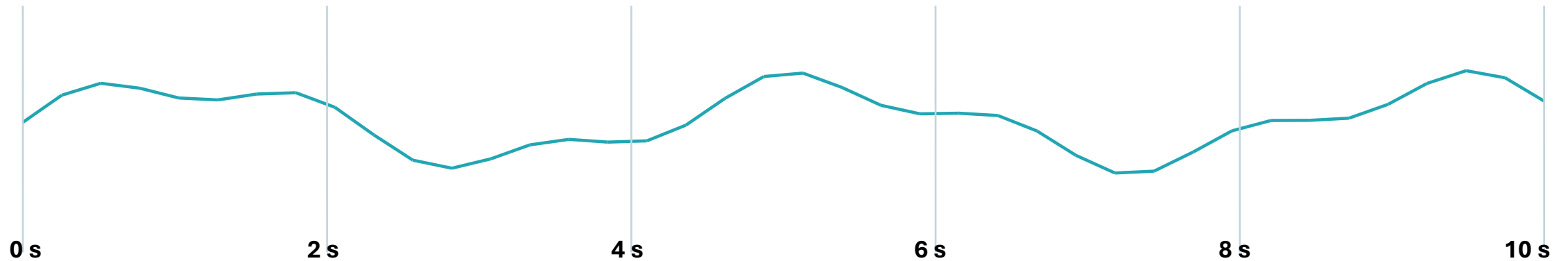
Diseño de muestreo

Elegimos una frecuencia suficiente para actividad humana y manejable por BLE.

$$f_s = 50 \text{ Hz}$$

$$T_s = 1/f_s = 20 \text{ ms}$$

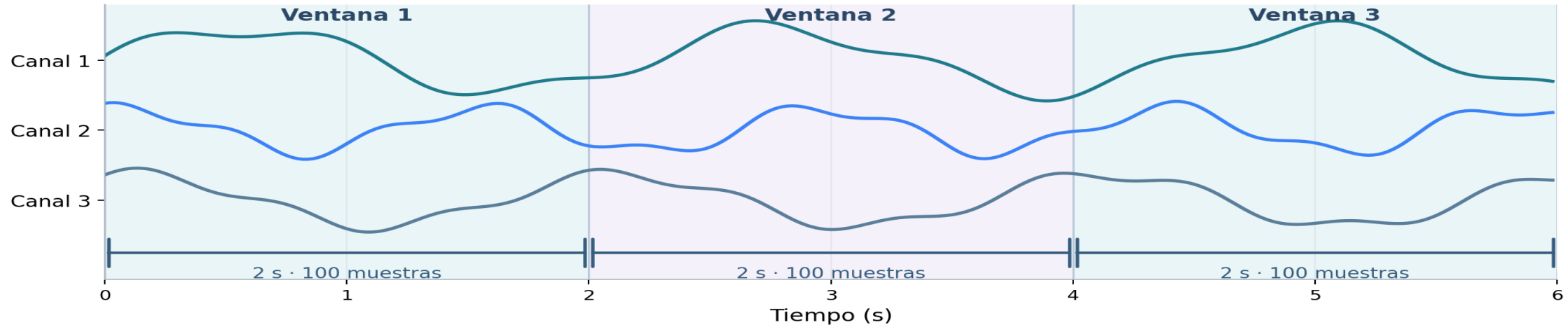
Ventana inicial: 2 s → 100 muestras



El mismo muestreo debe mantenerse en captura, entrenamiento e inferencia

Ventanas temporales

El modelo no clasifica muestras aisladas: clasifica fragmentos de movimiento.



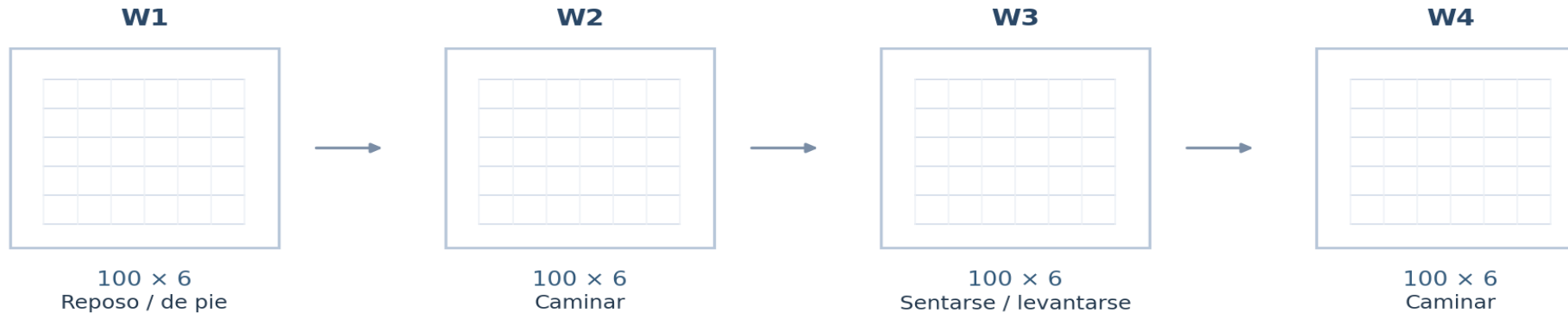
Ventana inicial: $2\text{ s} = 100\text{ muestras} \times 6\text{ canales}$

Cada ejemplo conserva la evolución temporal del movimiento en todos los canales del IMU.

Paso inicial recomendado: trabajar sin solapamiento para simplificar la construcción del dataset

Solapamiento temporal del 50 %

Una vez definido el tamaño de ventana, podemos aumentar el número de ejemplos desplazando la ventana en el tiempo.



Más ejemplos

Incrementa el número de ventanas disponibles para entrenamiento y prueba

Misma longitud

Se conserva la ventana de 2 s; únicamente cambia el desplazamiento temporal

Uso posterior

Conviene explorarlo después de validar el caso base sin solapamiento

¿Por qué Wi-Fi directo + UDP unicast?

El ESP32-S3 crea su propia red y envía las muestras directamente a la laptop.

Sin infraestructura

- ❖ No requiere router.
- ❖ No requiere internet.
- ❖ Funciona en un aula convencional.

Recepción estable

- ❖ UDP unicast evita pérdidas por broadcast.
- ❖ La laptop se registra con TINYWEAR_HELLO.
- ❖ Se verifica recepción y paquetes perdidos.

Suficiente para 50 Hz

- ❖ 6 canales IMU.
- ❖ ≈ 50 paquetes/s.
- ❖ Tráfico muy bajo para Wi-Fi.

Didáctico

- ❖ El participante ve red, captura, CSV y dataset en una misma interfaz.

TinyWear-1 → 192.168.4.1 → puerto UDP 5005 → Python → CSV

Paquete: 18 bytes por muestra

Transmisión binaria compacta para evitar problemas de longitud y mantener 50 Hz.

seq 2 B uint16	t_ms 4 B uint32	a_x 2 B int16	a_y 2 B int16	a_z 2 B int16	ω_x 2 B int16	ω_y 2 B int16	ω_z 2 B int16
-----------------------------	------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------	--------------------------------------

Ejemplo: 1025,20540,-0.934,-0.052,0.041,0.12,-0.08,0.04

Captura exacta

$N = \text{duración} \times 50 \text{ Hz}$. Para 20 s se guardan exactamente 1000 muestras

Calidad

$\text{Recepción} = N / (N + \text{paquetes perdidos}) \times 100$

Auditoría

seq permite detectar pérdidas reales durante la captura

Conexión ESP32-S3 ↔ MPU6050

Cuatro conductores; no requiere protoboard.

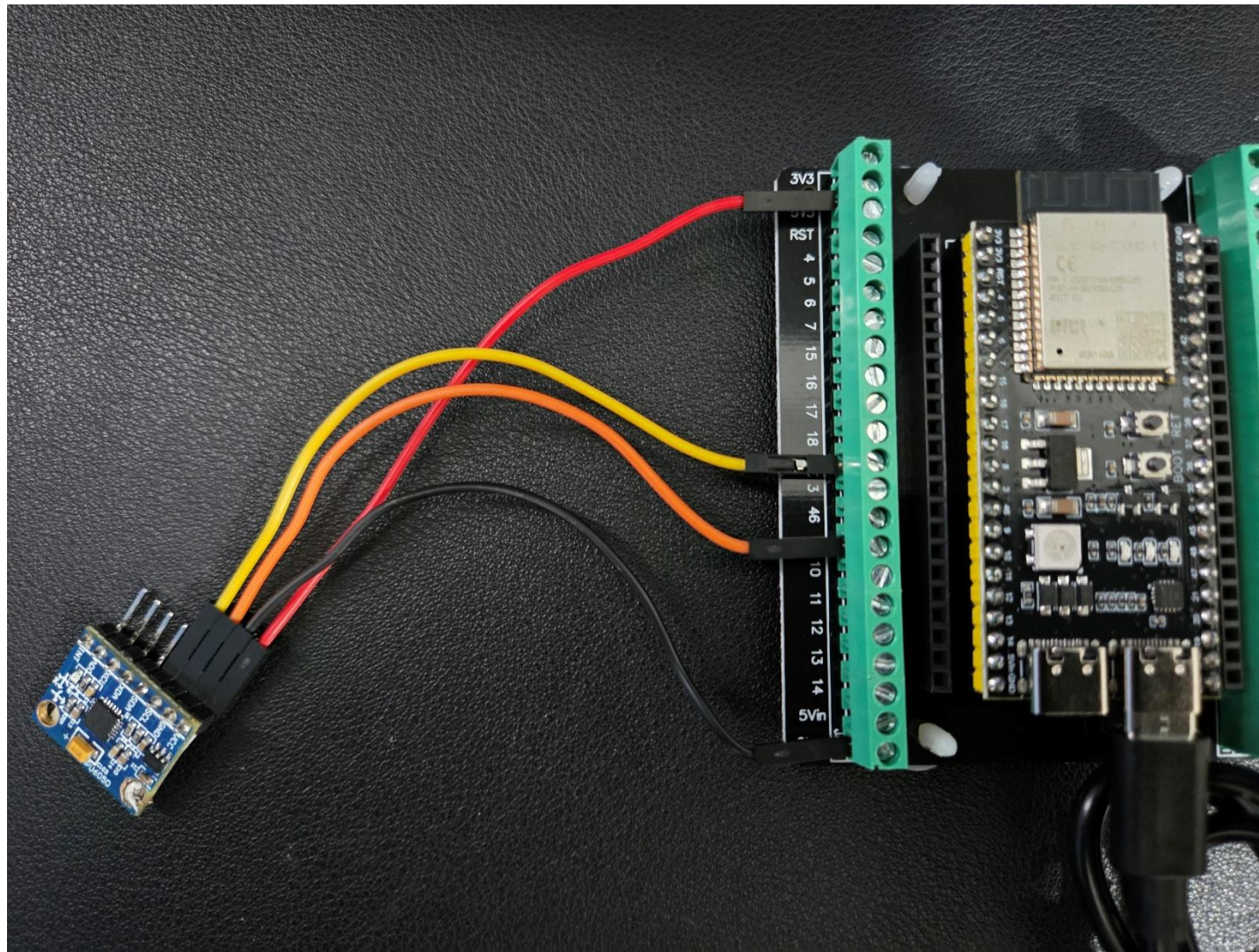
MPU6050	ESP32-S3
VCC	3.3 V
GND	GND
SDA	GPIO SDA
SCL	GPIO SCL

- ❖ En el código de referencia se usan GPIO 8/9 como punto de partida.
- ❖ Los pines I²C pueden variar según la tarjeta: verificar antes de energizar.
- ❖ El MPU6050 se configura a ± 4 g, ± 500 °/s y 50 Hz.

Recomendación operativa: entregar cada kit probado y con el cableado inmovilizado

Conexión ESP32-S3 ↔ MPU6050

Ejemplo de la conexión a realizar.



Flujo del firmware ESP32-S3

El microcontrolador mantiene una tarea sencilla y repetible durante la adquisición.



El ESP32-S3 transmite solo cuando la laptop se registra; esto evita pérdidas por broadcast

Firmware ESP32-S3

Ejemplo del código a utilizar.



```
P1C_TinyWear_WiFi_UDP_FINAL_V3 | Arduino IDE 2.3.6
File Edit Sketch Tools Help
ESP32S3 Dev Module
P1C_TinyWear_WiFi_UDP_FINAL_V3.ino
1  /*
2   TinyWear Wi-Fi / UDP UNICAST V2
3   ESP32-S3 + MPU6050
4   Taller ELECTRO 2026
5
6   Flujo:
7   MPU6050 -> ESP32-S3 -> Wi-Fi UDP unicast -> Python -> CSV
8
9   Red:
10  SSID: TinyWear-1
11  Password: TinyWear2026
12  ESP32: 192.168.4.1
13  Puerto UDP: 5005
14
15  Formato:
16  seq,t_ms,ax,ay,az,gx,gy,gz
17  Autor: Ernesto Sifuentes de la Hoya
18  */
19
20  #include <Arduino.h>
21  #include <Wire.h>
22  #include <WiFi.h>
23  #include <WiFiUdp.h>
24
25  #define KIT_ID 1
26
27  #define SDA_PIN 8
28  #define SCL_PIN 9
29  #define MPU_ADDR 0x68
30
31  const char* WIFI_PASSWORD = "TinyWear2026";
32
33  constexpr uint16_t UDP_PORT = 5005;
34  constexpr uint32_t FS_HZ = 50;
35  constexpr uint32_t SAMPLE_PERIOD_US = 1000000UL / FS_HZ;
36
37  constexpr float ACCEL_SCALE = 8192.0f; // +/-4 g
38  constexpr float GYRO_SCALE = 65.5f; // +/-500 dps
39
40  String ssid;
41  WiFiUDP udp;
```

Capturador Python Wi-Fi

La laptop se convierte en registrador de datos inalámbrico y organiza el dataset.

1. Registrar

Envía TINYWEAR_HELLO al ESP32-S3 y recibe ACK

2. Recibir

Escucha UDP unicast en el puerto 5005

3. Etiquetar

Selecciona sujeto, actividad, entrenamiento/prueba y duración

4. Guardar

Genera CSV con exactamente $N = \text{duración} \times 50$ muestras

TinyWear-1 → 192.168.4.1:5005 → **Capturador_Dataset_TinyWear_WIFI_FINAL_V3.py**

CSV: timestamp, ax, ay, az, gx, gy, gz

¿Qué facilita la interfaz?

Ordena el trabajo de captura y deja listo el dataset para entrenamiento y prueba.

Estructura automática del dataset

```
Dataset_TinyWear/  
├── entrenamiento/  
│   ├── reposo/  
│   ├── caminar/  
│   └── sentarse_levantarse/  
└── prueba/  
    ├── reposo/  
    ├── caminar/  
    └── sentarse_levantarse/
```

Control de captura

- ❖ Cuenta regresiva antes de iniciar
- ❖ Recepción BLE, muestras esperadas y paquetes perdidos

Etiquetado automático

- ❖ El usuario solo elige sujeto, actividad y destino
- ❖ El programa nombra y guarda cada archivo por repetición

Visualización

- ❖ Muestra la última captura del acelerómetro y del giroscopio
- ❖ Permite revisar rápidamente la calidad de la señal

Avance del dataset

- ❖ Cuenta por separado entrenamiento y prueba para cada clase
- ❖ Facilita cumplir la meta de muestras del taller

Salida de cada archivo CSV: timestamp, ax, ay, az, gx, gy, gz

Capturador Python

Ejemplo de la interfaz que utilizaremos para crear el dataset.

TinyWear - Capturador de Dataset Wi-Fi

ESP32-S3 + MPU6050 | Wi-Fi directo + UDP unicast | Reconocimiento de actividad humana

1. Comunicación Wi-Fi

Red: TinyWear-1
Password: TinyWear2026
UDP unicast: puerto 5005
Registrando laptop con TinyWear...

2. Configuración de captura

Participante

S01

Actividad

☒ Reposo / de pie

☐ Caminar

☐ Sentarse / levantarse

Destino

☒ Entrenamiento

☐ Prueba

Duración (s)

20

Cuenta regresiva (s)

3

Frecuencia

50 Hz (fija)

CAPTURAR MUESTRA

3. Carpeta del dataset

C:\Users\esifu\Downloads\TinyWear_WiFi

Cambiar carpeta

Abrir carpeta

4. Progreso del dataset

Meta entrenamiento/clase

5

Meta prueba/clase

2

Reposo / de pie: E 0/5 | P 0/2

Caminar: E 0/5 | P 0/2

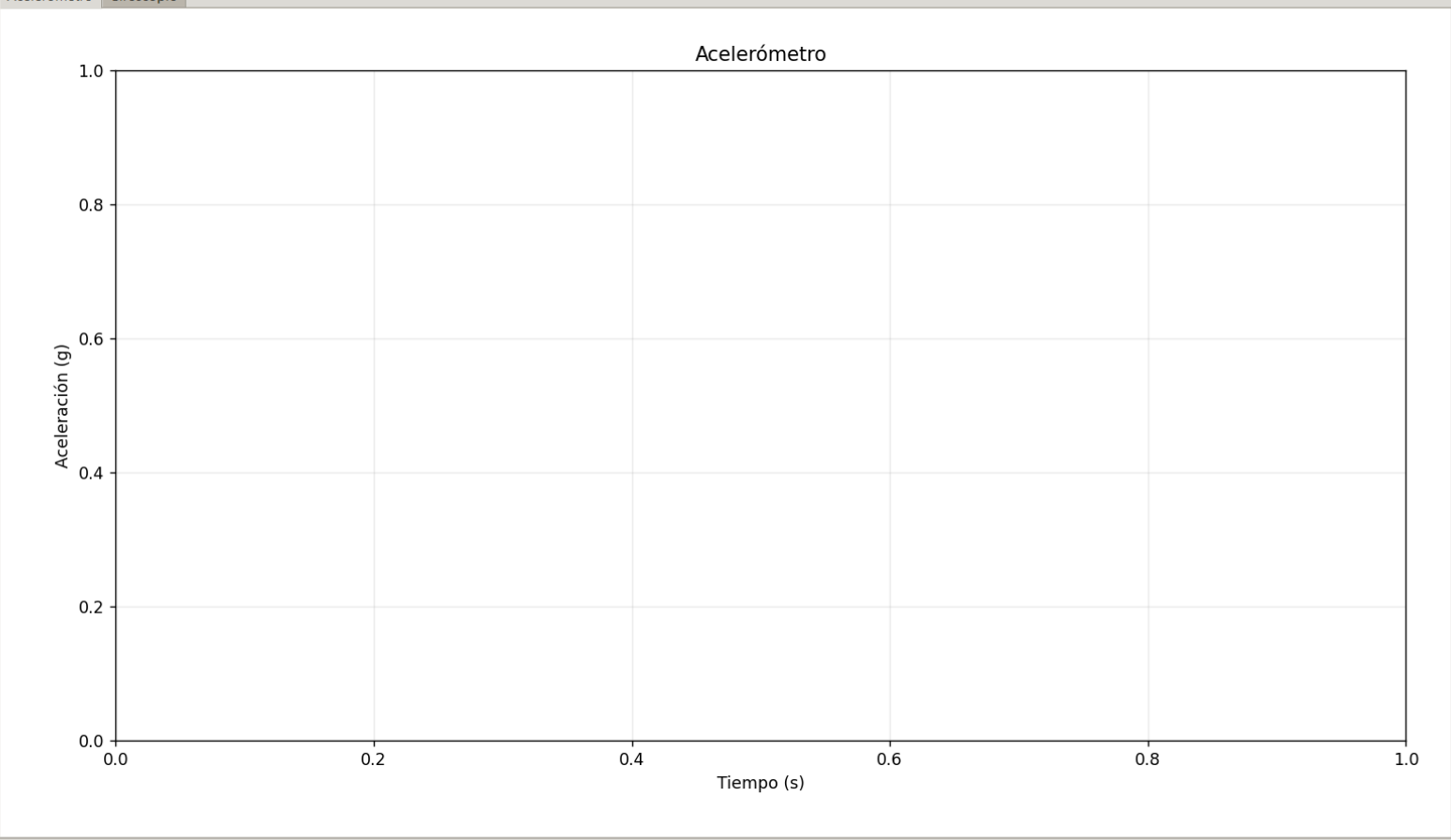
Sentarse / levantarse: E 0/5 | P 0/2

Última muestra capturada

Acelerómetro

Giroscopio

Acelerómetro



0 muestras

Conecte Windows a la red TinyWear-1.

Protocolo experimental: solo tres actividades

Menos clases → mejor control experimental → mayor probabilidad de una demo robusta.

REPOSO / DE PIE

- ❖ Permanecer de pie, postura natural y movimiento mínimo
- ❖ 2 capturas × 20 s por participante
- ❖ Objetivo: patrón de baja dinámica

CAMINAR

- ❖ Caminar a velocidad cómoda en un trayecto corto o circuito del salón
- ❖ 2 capturas × 20 s
- ❖ Objetivo: patrón cíclico

SENTARSE– LEVANTARSE

- ❖ Repetir ciclos controlados silla ↔ de pie, sin prisa
- ❖ 2 capturas × 20 s
- ❖ Objetivo: transiciones repetidas

Protocolo experimental de captura

Solo tres actividades: menos clases, mejor control, mayor robustez.



El dispositivo permanece en la misma posición y orientación durante todo el experimento.

1

Reposo / de pie



Permanecer de pie,
lo más quieto posible.

2

Caminar



Caminar a paso normal.

3

Sentarse / levantarse



Ritmo cómodo y continuo.

Secuencia
del experimento:

1

Reposo / de pie
(30 - 60 s)



2

Caminar
(30 - 60 s)



3

Sentarse / levantarse
(30 - 60 s)

Formato exacto del CSV

Cada archivo conserva tiempo, seis ejes y metadatos de la actividad.

timestamp | ax | ay | az | gx | gy | gz

Nombre de archivo

S01_caminar_001.csv

Estructura

Dataset_TinyWear/entrenamiento
/caminar/

Registro adicional

registro_capturas.csv guarda
recepción, pérdidas y metadatos

Regla del taller

Una captura = un archivo CSV. Para 20 s, cada archivo válido debe tener 1000 muestras

Adquisición y formato CSV

La captura debe producir datos útiles, no solo señales bonitas.



Paquete BLE (18 bytes)



Archivo CSV (formato del dataset)

subject	activity	repetition	seq	time_ms	ax_g	ay_g	az_g	gx_dps	gy_dps	gz_dps
01	reposo	1	0	0	0.012	0.003	1.006	0.8	-0.6	0.9
01	reposo	1	1	20	0.014	0.001	1.009	1.1	-0.4	0.7
01	reposo	1	2	40	0.010	-0.002	1.003	0.6	-0.8	1.0
01	caminar	1	0	0	0.152	0.041	0.982	12.3	-5.1	3.8
01	caminar	1	1	20	0.167	0.038	0.976	11.9	-4.7	4.1
01	caminar	1	2	40	0.141	0.036	0.989	13.5	-5.6	3.4
...	...	...	...	...	...	...	...	...	...	...



Ventajas

- sin cables
- movimiento libre
- captura en tiempo real



Control de calidad

- detección de paquetes perdidos
- etiquetado por actividad

TinyWear y Dataset

Módulo wearable
ESP32-S3 + MPU6050



Capturador Python



CSV



Modelo TinyML

- ❖ Sistema completo: IMU, ESP32-S3, Wi-Fi directo, Python, CSV y TinyML
- ❖ La laptop registra el dataset; el microcontrolador sigue siendo el centro del sistema
- ❖ La comunicación Wi-Fi/UDP unicast nos permite realizar capturas estables a 50 Hz

Cierre del día 1

Cada equipo debe terminar con evidencia reproducible, no solo con “señales bonitas”.

- ✓ TinyWear montado y estable en la cintura
- ✓ Conexión Wi-Fi funcional sin cable hacia la laptop
- ✓ Dos CSV de 20 s por actividad y por participante seleccionado
- ✓ Verificación visual de que las tres actividades producen patrones diferentes
- ✓ Registro del porcentaje aproximado de muestras recibidas y cualquier pérdida detectada

Objetivo cumplido: dataset base listo

Cierre del día 1

Cada equipo debe terminar con evidencia reproducible, no solo con “señales bonitas”.

Etapa	Trabajo práctico	Resultado verificable
Día 1: Instrumentar y adquirir	Montar TinyWear, observar los seis canales y capturar las tres actividades.	CSV etiquetados y revisados.
Día 2: Construir y entrenar	Revisar datos, separar entrenamiento/prueba, configurar ventanas y entrenar en Edge Impulse.	Modelo evaluado y matriz de confusión interpretada.
Día 3: Desplegar y validar	Exportar el modelo, ejecutar inferencia en ESP32-S3 y probar con otro participante.	Prototipo que reconoce actividades y registro de aciertos, errores y tiempo de respuesta.